

VERIFICATION PLAN

Parameterized Systolic Array Matrix Multiplication Accelerator

ECE-593: Fundamentals of Pre-Si Validation

Portland State University

Document Information

Field	Value
Document ID	VPL-SA-MM-001
Project / Product	Parameterized Systolic Array Matrix Multiplication Accelerator
Design Version	RTL v2.0 (Milestone 3 — Class Based Testbench)
Verification Level	Block-level (DUT: mm_accelerator_top)
Prepared By	Hanisha Dhananjaya, Venkat Sai Sumanth , Nikhil Swarna
Reviewed By	[Team Member — Design Lead]
Approved By	[Instructor — Venkatesh Patil]
Date Created	2026-04-20
Last Revised	2026-04-30
Revision	1.1 — (Milestone 3 Update)
Status	In Review
Confidentiality	Internal (ECE-593 Course Project)
Git Repository	https://github.com/hanisha369/Parameterized_Systolic_Array_Accelerator

Purpose Statement

This Verification Plan defines the strategy, environment, tools, and completion criteria for the functional verification of the Parameterized Systolic Array Matrix Multiplication Accelerator. The design implements an $N \times N$ output-stationary systolic array that computes $C = A \times B$ where A and B are $N \times N$ matrices of `DATA_WIDTH`-bit operands and C is an $N \times N$ matrix of `ACCUM_WIDTH`-bit results. The default configuration is $N=4$, `DATA_WIDTH=8`, `ACCUM_WIDTH=32`.

Milestone 2 extends verification using a **class-based SystemVerilog environment with coverage and scoreboard validation**

The plan is structured in accordance with the hierarchical verification methodology described in Comprehensive Functional Verification by Bruce Wile (Morgan Kaufmann).

1. Introduction

1.1 Purpose and Scope

This Verification Plan (VP) describes the complete approach for functionally verifying the Parameterized Systolic Array Matrix Multiplication Accelerator. It establishes the verification scope, methods, environment, required tools, completion criteria, and resource plan for the design under verification (DUV). The DUV is the top-level module `mm_accelerator_top`, composed of three sub-blocks: a controller FSM, an input-skew pipeline, and an $N \times N$ systolic array of processing elements (PEs).

This document applies to the following verification level:

- ☒ Block-level verification (`mm_accelerator_top` and sub-modules)
- ☐ Sub-system verification
- ☐ Full-chip / Top-level verification
- ☐ System-level verification

1.2 Design Overview

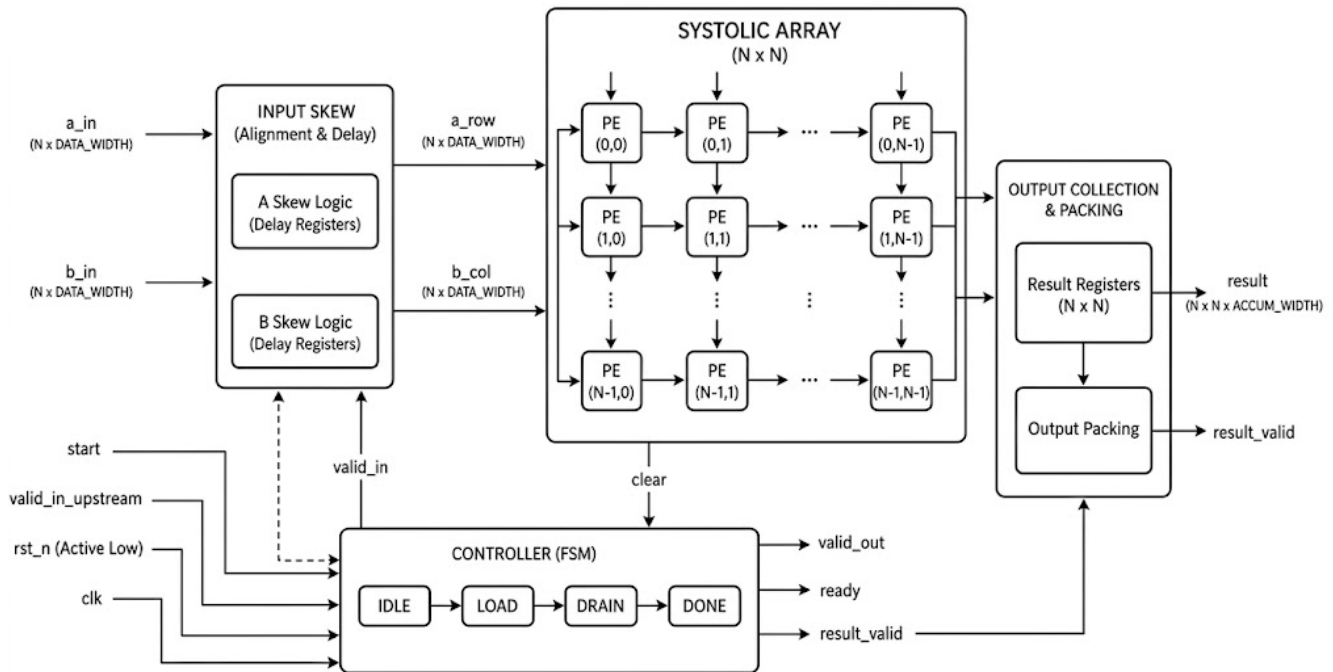
Project Description

The Parameterized Systolic Matrix Multiplication Accelerator is a scalable hardware design implemented in SystemVerilog to efficiently perform matrix multiplication using a parallel systolic array architecture. Matrix multiplication is a fundamental and compute-intensive operation used in applications such as machine learning, signal processing, and scientific computing, where high performance is essential.

The design utilizes an $N \times N$ grid of Processing Elements (PEs), where each PE performs a multiply-accumulate (MAC) operation. Data flows through the array in a structured manner: elements of matrix A propagate horizontally, while elements of matrix B propagate vertically. As data moves through the array, each PE computes partial products and accumulates results over multiple clock cycles, enabling high parallelism and throughput.

To ensure correct timing and alignment of inputs, an input skewing mechanism is implemented using delay registers. A controller based on a finite state machine (FSM) manages different phases of operation, including data loading, computation, and pipeline draining, while generating control signals such as `valid`, `ready`, and `result_valid`.

The accelerator is fully parameterized, allowing configuration of array size and data precision, making it adaptable to various performance requirements. A UVM-based verification environment is used to validate functionality through constrained random testing and comparison with a reference model. Overall, the design demonstrates an efficient and scalable approach to hardware-accelerated matrix multiplication.



1.3 Verification Hierarchy

Verification follows a hierarchical approach consistent with Bruce Wile's Comprehensive Functional Verification framework. Each level of the hierarchy addresses a specific set of functions.

Hierarchy Level	Design Scope	Covered In This VP	Referenced VP
Block	pe, input_skew, controller, systolic_array	☒ Yes	—
Sub-system	mm_accelerator_top	☒ Yes	—
Full-chip	N/A (standalone IP)	☐ No	N/A
System	N/A (standalone IP)	☐ No	N/A

1.4 References

Ref #	Document Title	Date
[R01]	High-throughput systolic array-based accelerator for hybrid transformer-CNN networks	2018-12-30
[R02]	High-throughput systolic array-based accelerator for hybrid transformer-CNN networks	2024-08

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
DUV / DUT	Design Under Verification / Design Under Test — mm_accelerator_top

PE	Processing Element — single MAC unit in the systolic array
MAC	Multiply-Accumulate operation: $\text{acc} \leftarrow \text{acc} + a*b$
FSM	Finite State Machine (controller module)
UVM	Universal Verification Methodology (Accellera / IEEE 1800.2)
SV / SVA	SystemVerilog / SystemVerilog Assertions
TB	Testbench
FC / CC	Functional Coverage / Code Coverage
RAL	Register Abstraction Layer (UVM register model)
N	Array dimension parameter (default 4 $\rightarrow$ 4×4 array, 16 PEs)
DATA_WIDTH	Bit-width of A/B operands (default 8)
ACCUM_WIDTH	Bit-width of accumulator and result element (default 32)
DRAIN_CYCLES	Controller parameter — 2*N cycles of pipeline drain after last input

2. Functions to Be Verified

This section lists all functions and features of the DUV within scope of this verification plan. Each function is assigned a unique identifier for traceability (Wile, Chapters 3-4).

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-001	Reset behavior — all registers (PE accum, skew delays, controller FSM, counters) reset to 0 on active-low rst_n	pe.sv, controller.sv, input_skew.sv	Critical	Directed + Assertion
FV-002	Controller FSM transitions — IDLE→LOAD on start, LOAD→DRAIN after N valid inputs, DRAIN→DONE after DRAIN_CYCLES, DONE→IDLE after one cycle	controller.sv	Critical	Directed + FSM Coverage
FV-003	Start-pulse handshake — start pulse in IDLE asserts clear for one cycle and arms the pipeline	controller.sv	Critical	Directed + Assertion
FV-004	Input skew correctness — lane i delayed by exactly i cycles before entering the systolic array	input_skew.sv	Critical	Directed + Scoreboard
FV-005	PE MAC operation — accum ← accum + a_in*b_in when valid_in, hold on !valid_in, clear to 0 on clear	pe.sv	Critical	Directed + Constrained-Random
FV-006	PE data forwarding — a_in → a_out and b_in → b_out registered by one clock	pe.sv	High	Directed + Assertion
FV-007	Valid-signal diagonal propagation — valid_diag[i][j] follows the systolic diagonal with 1-cycle delay per hop	systolic_array.sv	High	Directed + Assertion
FV-008	Full matrix multiplication correctness — $result[i][j] == \sum A[i][k]*B[k][j]$ for all (i,j) and all valid operand ranges	All modules	Critical	Scoreboard vs. reference model
FV-009	Result-valid strobe — result_valid pulses exactly one cycle and result bus is stable when asserted	controller.sv, mm_accelerator_top.sv	Critical	Directed + Assertion
FV-010	Back-to-back operation — after DONE, a new start pulse cleanly begins a fresh matmul with reset accumulators	controller.sv + pe.sv (clear)	High	Directed + Random
FV-011	Corner values — A=0, B=0, A=Identity, B=Identity, A=all-ones, B=all-ones, max operand (0xFF)	mm_accelerator_top.sv	Critical	Directed (existing 4 tests)

FV-012	Accumulator overflow boundary — largest legal product sum $\leq 2^{32}-1$ at DATA_WIDTH=8, N=4 (max = $4 \cdot 255 \cdot 255 = 260,100$) fits in 32 bits	pe.sv	Medium	Directed
FV-013	Parameterization — design compiles and functions correctly for $N \in \{2,4,8\}$, $DATA_WIDTH \in \{4,8,16\}$	All modules	High	Regression Matrix
FV-014	valid_in_upstream de-assertion mid-transaction — LOAD stalls until re-asserted, no spurious accumulation	controller.sv	Medium	Directed + Random
FV-015	Result bus stability — result register holds last-computed value until next DONE	mm_accelerator_top.sv	Medium	Assertion

Priority legend: Critical = must pass before sign-off; High = required for Milestone 5; Medium = best-effort; Low = stretch goal.

3. Functions Not Covered at This Verification Level

NC-ID	Function / Feature
NC-001	Assertions
NC-002	UVM Based verification

4. Verification Strategy and Methods

4.1 Overall Strategy

The verification strategy is a layered approach that scales across the five course milestones:

- Milestone 1 (Traditional TB): Directed tests with a SystemVerilog reference model — already implemented in `tb_mm_accelerator.sv`. 4 directed tests running today.
- Milestones 2-3 (Class-based TB): Transition to class-based stimulus generation, transactor/driver/monitor separation, scoreboard class.
- Milestones 4-5 (UVM TB): Full UVM 1.2 environment with agents, sequences, coverage collectors, and constrained-random regression (corner, random, stress, full regression).
- Throughout: Assertion-based verification (SVA) bound to each sub-module to catch protocol violations immediately.
- Throughout: Functional coverage groups mapped 1:1 to the FV-IDs of Section 2.

Milestone 3 Update

- Transitioned from traditional TB - **class-based verification**
- Implemented full verification flow:
 - Transaction → Generator → Driver → Monitor → Scoreboard → Coverage
- 20 transactions per run (directed + randomized)
- Burst ID tracking for traceability
- Coverage-driven verification using VCS

4.2 Testbench Architecture

Milestone 1 uses a single-module SystemVerilog testbench (`tb_mm_accelerator.sv`) that instantiates `mm_accelerator_top`, generates a 100 MHz clock (`CLK_PERIOD=10ns`), applies reset, pulses start, feeds N cycles of A and B, waits on `result_valid`, unpacks the result bus, and compares against a software reference model (`ref_model` function). A timeout watchdog (10,000 cycles) kills stalled simulations. Milestones 4-5 will migrate to a UVM environment with per-interface agents.

Milestone 3 Architecture

The testbench is implemented using a **class-based SystemVerilog environment (non-UVM)**.

4.2.1 Testbench Components

Component	Description	Status / Reuse
<code>tb_mm_accelerator</code> (top)	Instantiates DUV, clock/reset, drives stimulus, checks results	implemented (M1)
<code>ref_model()</code> function	Golden SW reference: $C[i][j] = \sum A[i][k] \cdot B[k][j]$	implemented (M1)
<code>pack_row</code> / <code>pack_col</code>	Helper functions to pack 2-D A/B matrices into flat N·DATA_WIDTH buses	implemented (M1)
<code>unpack_result</code>	Unpacks $N^2 \cdot \text{ACCUM_WIDTH}$ result bus into 2-D array for checking	implemented (M1)
<code>run_test</code> task	Parameterized test runner — reset, drive, wait, check, report PASS/FAIL	implemented (M1)
UVM Test (<code>uvm_test</code>)	Planned for M4-M5 — controls <code>mm_corner/rand/stress/regression</code> scenarios	Planned (M4)
UVM Env (<code>uvm_env</code>)	Planned — assembles agents, scoreboard, coverage	Planned (M4)

Agent — matrix_agent	Planned — driver + monitor + sequencer for A/B input interface	Planned (M4)
Scoreboard	Planned — UVM scoreboard wrapping ref_model()	Planned (M4)
SVA Assertions (bind)	Planned — protocol, valid-stability, FSM-reach properties	Planned (M4)
Register Model (RAL)	Not applicable — no memory-mapped register interface in DUV	N/A
Transaction (Class Based)	Stores A, B, expected C, actual C, burst_id	Implemented MS3
Generator (Class Based)	Generates 20 transactions (directed + random)	Implemented MS3
Driver (Class Based)	Drives DUT inputs via interface	Implemented MS3
Input Monitor (Class Based)	Observes input handshake	Implemented MS3
Output Monitor (Class Based)	Captures result_valid output	Implemented MS3
Scoreboard (Class Based)	Compares expected vs actual	Implemented MS3
Coverage (Class Based)	Functional coverage collection	Implemented MS3
Environment (Class Based)	Connects all components	Implemented MS3

4.3 Stimulus Methods

4.3.1 Directed Tests

The four directed tests already implemented in tb_mm_accelerator.sv are:

- TEST 1 — All-Zeros: A=0, B=0, expected result = all zeros. Verifies clear-path and baseline accumulation.
- TEST 2 — Identity × Identity: A=I, B=I, expected result = I. Verifies diagonal data flow and PE indexing.
- TEST 3 — All-Ones: A=[1..1], B=[1..1], expected result element = N. Verifies full MAC path on every PE.
- TEST 4 — Known Random Values: hand-crafted A, B verified against ref_model. Verifies arbitrary arithmetic.

4.3.2 Constrained-Random Tests (Milestones 3)

- rand_test: Randomize A and B with constraint: each element $\in [0, 2^{\text{DATA_WIDTH}}-1]$. Weighted bins to hit corner values (0, 1, MAX) more often.
- corner_test: Directed-randomization focused on boundary patterns — single-nonzero matrices, sparse matrices, diagonal-only, max-operand.
- stress_test: N back-to-back matmuls with randomized inter-arrival gaps; confirms pipeline can be re-armed cleanly.
- regression_test: Composite test that invokes corner + rand + stress sequences with a fixed seed list for reproducibility.

- Coverage-driven re-seed: when a covergroup bin is short, generate additional stimulus biased toward it.

4.4 Checking Strategy

- Scoreboard (ref_model) — for each completed matmul, compare got[i][j] vs. expected[i][j] for all N^2 elements; print MISMATCH on any diff.
- End-of-sim report — PASS/FAIL per test, total mismatches, covergroup percentages.
- Waveform dump (FSDB) — \$fsdbDumpvars(0, tb_mm_accelerator) for post-mortem debug in Verdi / Simvision.

5. Verification Environment

5.1 Simulation Environment

Item	Value
Simulator	Synopsys VCS
Simulator Version	V-2023.12-SP2
Verification Methodology (M3)	Class-based SystemVerilog
Verification Methodology (M4-M5)	UVM 1.2 (Accellera / IEEE 1800.2)
HDL Language	SystemVerilog IEEE 1800-2017
Assertion Language	SystemVerilog Assertions (SVA)
Compile/Sim Options (M1)	-sv -uvm -access +rwc -input restart.tcl
Waveform Format	FSDB (Verdi) and SHM (Simvision) — dumped via \$fsdbDumpvars
Seed Management	Fixed seed for regression reproducibility; random seeds for exploration
Regression Framework	GNU Make (targets: sim, corner, rand, stress, regression, clean)
Coverage Database	cov_work/ (Xcelium IMC coverage database)
Version Control	Git — GitHub repository (see Document Information)
CI / Build Automation	Makefile + scripts/run_sim.sh, scripts/run_uvm.sh

5.2 Coverage Plan

Each FV-ID from Section 2 maps to a covergroup. Targets align with Milestone 5 sign-off.

FV-ID	Covergroup	Bins / Cross Points	Target	Tool
FV-004 / FV-005	cg_a_range	A-input operand range during valid input drive	100%	VCS
FV-004 / FV-005	cg_b_range	B-input operand range during valid input drive	100%	VCS
FV-008 / FV-011	cg_result	Output result distribution when result_valid is asserted	100%	VCS
FV-002 / FV-003 / FV-009	cg_fsm	Control behavior of ready, result_valid, and start	100%	VCS

6. Required Tools

Category	Tool / Package	Version	Vendor	Status
----------	----------------	---------	--------	--------

Simulator	VCS	23.09	Synopsys	Licensed (PSU lab)
Waveform Viewer	Verdi / Simvision	2023	Synopsys / Cadence	Licensed (PSU lab)
Coverage Tool	VCS	23.09	VCS	Licensed
UVM Library	UVM 1.2 Accellera distribution (shipped with Xcelium)	1.2	Accellera	Available
Build / Regression	GNU Make + bash scripts (scripts/run_sim.sh, run_uvm.sh)	GNU Make 4.x	Open source	Available
Version Control	Git + GitHub	Git 2.x	Open source	Available
Documentation	Markdown + Word (.docx) + GitHub README	—	—	Available
Bug Tracking	GitHub Issues	—	GitHub	Available
Reference Model	SystemVerilog function (ref_model in TB)	SV 2017	Internal	Implemented

7. Completion Criteria

Verification is declared complete when ALL measurable criteria below are satisfied (Wile, Chapter 13). Sign-off is by the Verification Lead and the course instructor.

7.1 Simulation Completion Criteria

#	Criterion	Target	Achieved
1	All directed tests pass with zero unexpected failures	100% pass	[100] / [%]
2	Constrained-random regression passes across N seeds	$N \geq 10$ seeds	[Y] / [N]
3	All functional covergroups close	$\geq 95\%$	[100] / [%]
4	Statement + branch code coverage	$\geq 90\%$	[100] / [%]
5	Toggle coverage on DUV outputs and interfaces	$\geq 65\%$	[70] / [%]
6	FSM state + transition coverage (controller)	100%	[100] / [%]
7	All SVA assertions pass in final regression	0 failures	[] / [N]
8	No Critical or High severity bugs open	0 open	[] / [N]
9	Coverage waivers documented and approved	On record	[x] Done

Note:

The toggle coverage is not 100% because the design has **wide datapath buses and repeated systolic-array internal signals**, and the 20 directed/random transactions do not force every single bit to toggle both ways, **0→1 and 1→0**. Some accumulator/output bits, especially higher-order bits, may never change because the tested matrix values do not naturally produce results large enough to exercise every bit. This does not indicate a functional failure because line, branch, condition, FSM, and functional coverage are closed, and the scoreboard confirms correct results for all transactions.

7.2 Formal Verification Completion Criteria (Stretch)

#	Criterion	Target	Status
1	Controller FSM reachability + deadlock-free	PROVEN	[] Done
2	Input-skew per-lane delay property	PROVEN or BOUNDED	[] Done
3	All formal counterexamples analyzed	0 open CEX	[] Done

7.3 Sign-Off Checklist

Sign-Off Item	Responsible Party	Date	Sign
Simulation completion criteria met (7.1)	Verification Lead	[Date]	___
Formal criteria met (7.2) — if pursued	Verification Lead	[Date]	___
Coverage waivers approved	Design Lead	[Date]	___
All Critical/High bugs closed or deferred	Design Lead	[Date]	___
Verification plan reviewed and up-to-date	Verification Lead	[Date]	___

Regression results archived in Git	Verification Lead	[Date]	___
Final sign-off — course deliverable	Instructor	[Date]	___

8. Resources and Schedule

8.1 Human Resources

Team structure for the 5-milestone course project. Fill in names before M1 submission.

Role	Name / Team Member	Allocation	Responsibilities
Verification Engineer (TB)	Hanisha	33%	Driver, Interface, Input monitor, Test
Verification Engineer (TB)	Sai Sumanth	33%	generator, Output monitor, environment,
Verification Engineer (TB)	Nikhil	33%	Scoreboard, Transaction, Top
Instructor / Approver	Venkatesh Patil	—	Milestone review, sign-off

8.2 Hardware Resources

Resource	Specification	Quantity	Availability
PSU linux lab servers	Multi-core CPU, 32 GB RAM per host	Shared	☒ Available
VCS/ Verdi licenses	Floating license pool	Shared	☒ Available
GitHub repository	Public GitHub repo, unlimited CI minutes for public repos	1	☒ Available
Personal laptops	For documentation / editing	3	☒ Available

8.3 Schedule

Five-milestone schedule aligned with ECE-593 course calendar (replace [MM/DD] with actual dates).

#	Milestone / Deliverable	Planned Start	Planned End	Owner	Status
M1	Design spec, Verification Plan, RTL, Traditional TB (this document)	[04/11]	[04/19]	All	☒ In Progress
M2	Class-based TB (CLASS_TB),	[04/24]	[04/29]	V-Eng	☐ Done
M3	Class-based TB with scoreboard + basic coverage	[04/27]	[04/29]	V-Eng	☐ Done
M4	UVM environment — agents, sequences, uvm_env	[MM/DD]	[MM/DD]	V-Eng	☐ Open
M5	UVM regression suite, coverage closure, final sign-off	[MM/DD]	[MM/DD]	V-Lead	☐ Open

M6	Final deliverable — FINAL/ directory with all milestones	[MM/DD]	[MM/DD]	V-Lead	☐ Open
----	---	---------	---------	--------	-------------------------------

9. Risk and Mitigation

#	Risk Description	Likelihood	Impact	Mitigation Plan
R1	Toggle Coverage Low	High	Low	Waived
R2	RTL instability delaying testbench development	High	High	RTL is already frozen and working (all 4 tests pass). Testbench developed against stable mm_accelerator_top interface with fixed port list

10. Document Revision History

Revision	Date	Author	Description of Change	Approved By
0.1	2026-04-20	[Team — V-Lead]	Initial draft populated from RTL source files (pe.sv, systolic_array.sv, mm_accelerator_top.sv, input_skew.sv, controller.sv, tb_mm_accelerator.sv)	—
1.1	[2026-04-29]	[Team]	Added class-based TB, coverage, transcript	[Instructor]